

Emotion Detection from scratch

Author: Abhishek Kumar

Date created: 2026/04/29

Description: Training an image classifier from scratch on the Kaggle FER-2013 Dataset.

Data Set-

Kaggle - <https://www.kaggle.com/datasets/msambare/fer2013> The data consists of 48x48 pixel grayscale images of faces. The faces have been automatically registered so that the face is more or less centred and occupies about the same amount of space in each image.

The task is to categorize each face based on the emotion shown in the facial expression into one of seven categories (0=Angry, 1=Disgust, 2=Fear, 3=Happy, 4=Sad, 5=Surprise, 6=Neutral)

Setting up Kaggle Directory

```
In [2]: !mkdir -p ./kaggle
        !cp kaggle.json ~/.kaggle/
```

```
In [3]: !kaggle datasets download -d msambare/fer2013
```

```
Warning: Your Kaggle API key is readable by other users on this system! To fix this,
you can run 'chmod 600 /root/.kaggle/kaggle.json'
Downloading fer2013.zip to /content
 94% 57.0M/60.3M [00:01<00:00, 44.0MB/s]
100% 60.3M/60.3M [00:02<00:00, 30.8MB/s]
```

```
In [ ]: !unzip /content/fer2013.zip -d /content/
```

Installing Packages

```
In [5]: import os
        import random
        import numpy as np
        import pandas as pd
        import matplotlib.pyplot as plt
        import cv2
        import imghdr
        import tensorflow as tf
        from PIL import Image
        import seaborn as sns
```

```

from sklearn.metrics import confusion_matrix, classification_report, roc_curve, auc
from sklearn.utils.class_weight import compute_class_weight

from tensorflow.keras import layers, models, regularizers, optimizers
from tensorflow.keras.applications import VGG16, ResNet50V2
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping, ReduceLROnPl
from tensorflow.keras.layers import Conv2D, MaxPooling2D, BatchNormalization, Dropo
from tensorflow.keras.models import Sequential, Model
from tensorflow.keras.optimizers import Adam, Adamax
from tensorflow.keras.preprocessing.image import ImageDataGenerator, load_img, img_
from keras.utils import plot_model

```

Setting up Directory to save model data.

```
In [6]: os.makedirs('test_dir', exist_ok=True)
```

```
In [7]: # Define your project name
project_name = 'FER_2013_Emotion_Detection'

# List your models
model_names = [
    'Custom_CNN_From_Scratch',
    'Custom_CNN_With_Augmentation',
    'VGG16_Transfer_Learning',
    'ResNet50_Transfer_Learning'
]

# Base directory (in this case, your Google Colab workspace)
base_dir = '/content/'

# Create the project directory
project_dir = os.path.join(base_dir, project_name)
os.makedirs(project_dir, exist_ok=True)

# Create a subdirectory for each model
for each_model in model_names:
    model_dir = os.path.join(project_dir, each_model)
    os.makedirs(model_dir, exist_ok=True)
    # Example subdirectories for model-related files
    # os.makedirs(os.path.join(model_dir, 'checkpoints'), exist_ok=True)
    # os.makedirs(os.path.join(model_dir, 'logs'), exist_ok=True)
    # os.makedirs(os.path.join(model_dir, 'saved_models'), exist_ok=True)

print(f'Project directory structure created at: {project_dir}')
```

Project directory structure created at: /content/FER_2013_Emotion_Detection

Data Cleaning

```
In [ ]: # Define the list of acceptable image extensions
image_exts = ['.jpeg', '.jpg', '.png']
```

```

# Path to the directory containing image classes and possibly other nested subdirec
data_dir = '/content/train'

# Walk through all directories and files in the dataset
for root, dirs, files in os.walk(data_dir):
    for file in files:
        # Construct the path to the current file
        file_path = os.path.join(root, file)

        try:
            # Check the file type of the current file
            file_type = imghdr.what(file_path)

            # If the file extension is not in the allowed list, remove it
            if file_type not in image_exts:
                print(f'Image not in ext list {file_path}')
                os.remove(file_path)
            else:
                # Proceed to process the image if needed, for example, reading it w
                img = cv2.imread(file_path)

        except Exception as e:
            # Print out the issue and the path of the problematic file
            print(f'Issue with file {file_path}. Error: {e}')
            # Optionally, remove files that cause exceptions
            os.remove(file_path)

```

Data Analysis

```

In [8]: # Define a function to count the number of files (assumed to be images for this con
# The function returns a DataFrame with these counts, indexed by a specified set na
def count_files_in_subdirs(directory, set_name):
    # Initialize an empty dictionary to hold the count of files for each subdirecto
    counts = {}

    # Iterate over each item in the given directory.
    for item in os.listdir(directory):
        # Construct the full path to the item.
        item_path = os.path.join(directory, item)

        # Check if the item is a directory.
        if os.path.isdir(item_path):
            # Count the number of files in the subdirectory and add it to the dicti
            counts[item] = len(os.listdir(item_path))

    # Convert the counts dictionary to a DataFrame for easy viewing and analysis.
    # The index of the DataFrame is set to the provided set name.
    df = pd.DataFrame(counts, index=[set_name])
    return df

# Paths to the training and testing directories.
train_dir = '/content/train'
test_dir = '/content/test'

```

```

# Count the files in the subdirectories of the training directory and print the res
train_count = count_files_in_subdirs(train_dir, 'train')
print(train_count)

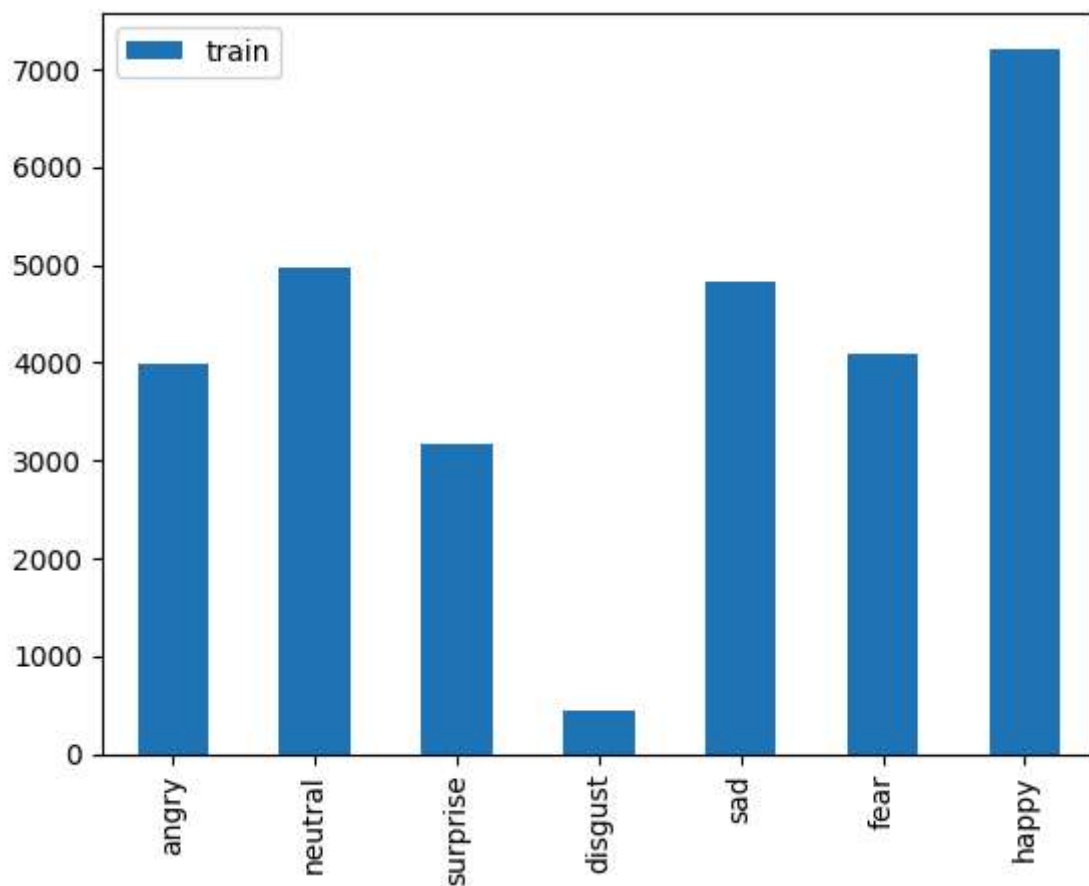
# Count the files in the subdirectories of the testing directory and print the resu
test_count = count_files_in_subdirs(test_dir, 'test')
print(test_count)

```

	sad	fear	happy	disgust	neutral	angry	surprise
train	4830	4097	7215	436	4965	3995	3171
	sad	fear	happy	disgust	neutral	angry	surprise
test	1247	1024	1774	111	1233	958	831

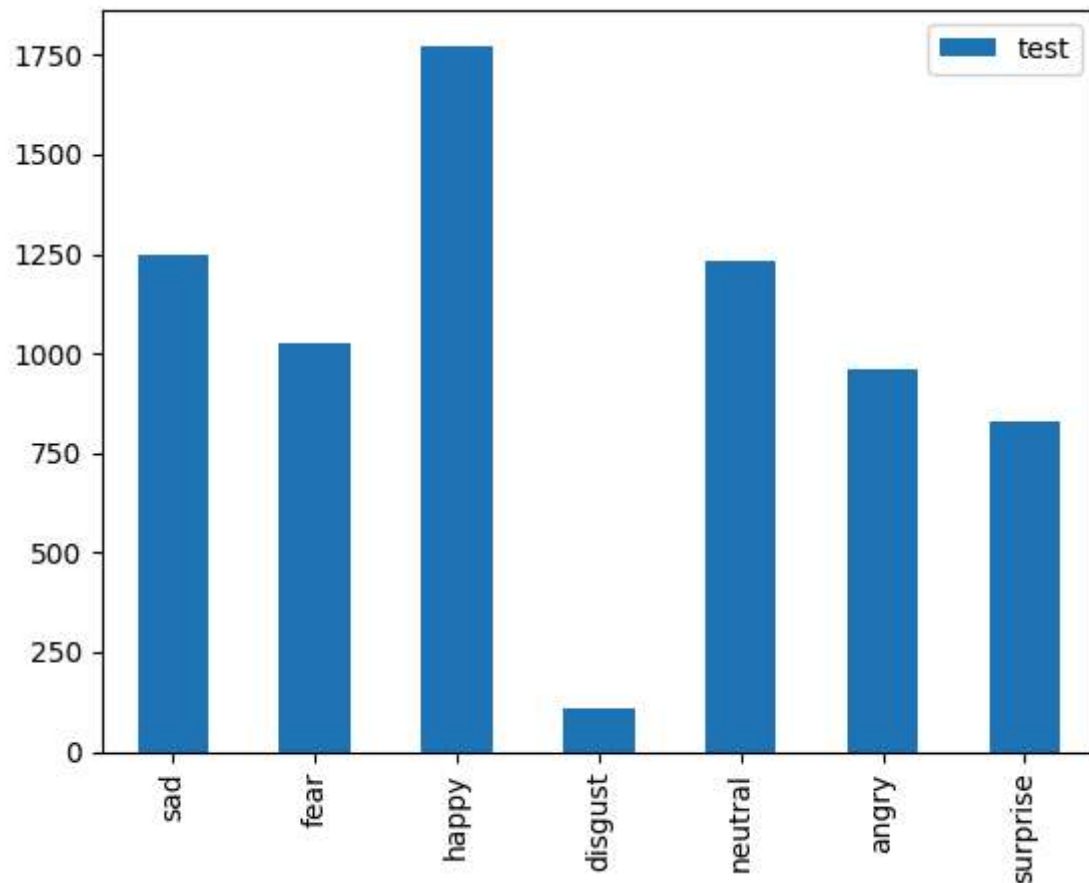
```
In [ ]: train_count.transpose().plot(kind='bar')
```

Out[]: <Axes: >



```
In [9]: test_count.transpose().plot(kind='bar')
```

Out[9]: <Axes: >



Plotting one image from each emotion.

```
In [10]: emotions = os.listdir(train_dir)
plt.figure(figsize=(15,10))

for i, emotion in enumerate(emotions, 1):
    folder = os.path.join(train_dir, emotion)
    img_path = os.path.join(folder, os.listdir(folder)[42])
    img = plt.imread(img_path)
    plt.subplot(3, 4, i)
    plt.imshow(img, cmap='gray')
    plt.title(emotion)
    plt.axis('off')
```



Plotting random images from given directory

```
In [12]: def plot_images_from_directory(directory_path, class_name, num_images=9):
# Retrieve list of all file names in the directory
image_filenames = os.listdir(directory_path)

# If there are fewer images than requested, we'll just show them all
if len(image_filenames) < num_images:
    print(f"Only found {len(image_filenames)} images in {directory_path}, displ
    num_images = len(image_filenames)

# Randomly select 'num_images' number of file names
selected_images = random.sample(image_filenames, num_images)

# Plotting the images
fig, axes = plt.subplots(3, 3, figsize=(5, 5)) # Adjust the size as needed
axes = axes.ravel()

for i, image_file in enumerate(selected_images):
    image_path = os.path.join(directory_path, image_file)
    # image = Image.open(image_path)
    image = load_img(image_path)
    axes[i].imshow(image)
    axes[i].set_title(f"Image: {class_name}")
    axes[i].axis('off') # Hide the axis

plt.tight_layout()
plt.show()
```

```
In [13]: # Placeholder for the directory path
angry_directory_path = '/content/train/angry' # Replace with your directory path
plot_images_from_directory(angry_directory_path, class_name = 'Angry')
```

Image: Angry



Image: Angry



Image: Angry



Image: Angry



Image: Angry



Image: Angry



Image: Angry



Image: Angry



Image: Angry



```
In [14]: # Placeholder for the directory path
disgust_directory_path = '/content/train/disgust' # Replace with your directory pa
plot_images_from_directory(disgust_directory_path, class_name = 'Disgust')
```

Image: Disgust



Image: Disgust



Image: Disgust



Image: Disgust



Image: Disgust



Image: Disgust



Image: Disgust



Image: Disgust



Image: Disgust



```
In [15]: # Placeholder for the directory path
surprise_directory_path = '/content/train/surprise' # Replace with your directory
plot_images_from_directory(surprise_directory_path, class_name = 'Surprise')
```

Image: Surprise



Image: Surprise



Image: Surprise



Image: Surprise

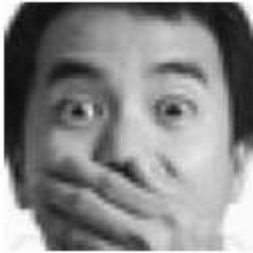


Image: Surprise



Image: Surprise



Image: Surprise



Image: Surprise



Image: Surprise



Checking shapes and channels.

```
In [16]: image = '/content/train/angry/Training_10118481.jpg'

import cv2

img = cv2.imread(image) # Default Load in color format.

# If the image is loaded successfully, print its pixel values
if img is not None:
    # print(img)
    print("Shape:", img.shape)
else:
    print("The image could not be loaded. Please check the path and file permission")
```

Shape: (48, 48, 3)

```
In [17]: import cv2

image_path = '/content/train/angry/Training_10118481.jpg'

# Load the image in grayscale
img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
```

```
# If the image is loaded successfully, print its pixel values
if img is not None:
    # print(img)
    print("Shape:", img.shape) # This should now print (48, 48)
else:
    print("The image could not be loaded. Please check the path and file permission")
```

Shape: (48, 48)

Model 1 : Custom CNN from Scratch

Initializing the ImageGenerators

```
In [18]: # Define paths to the train and validation directories
train_data_dir = '/content/train'
test_data_dir = '/content/test'
```

```
In [19]: # Set some parameters
img_width, img_height = 48, 48 # Size of images
batch_size = 64
epochs = 10
num_classes = 7 # Update this based on the number of your classes
```

```
In [20]: # Rescale the pixel values (0-255) to the [0, 1] interval
data_generator = ImageDataGenerator(rescale=1./255,
                                     validation_split=0.2)

# Automatically retrieve images and their classes for train and validation sets
train_generator = data_generator.flow_from_directory(
    train_data_dir,
    target_size=(img_width, img_height),
    batch_size=batch_size,
    class_mode='categorical',
    color_mode='grayscale',
    subset='training')

validation_generator = data_generator.flow_from_directory(
    train_data_dir,
    target_size=(img_width, img_height),
    batch_size=batch_size,
    class_mode='categorical',
    color_mode='grayscale',
    subset='validation')

test_generator = data_generator.flow_from_directory(
    test_data_dir,
    target_size=(img_width, img_height),
    batch_size=batch_size,
    class_mode='categorical',
    color_mode='grayscale',)
# subset='validation')
```

Found 22968 images belonging to 7 classes.

Found 5741 images belonging to 7 classes.

Found 7178 images belonging to 7 classes.

```
In [21]: # Accessing class labels for the training data
train_class_labels = train_generator.class_indices
print("Training class labels:", train_class_labels)

# Accessing class labels for the validation data
validation_class_labels = validation_generator.class_indices
print("Validation class labels:", validation_class_labels)

# Accessing class labels for the validation data
test_class_labels = test_generator.class_indices
print("Validation class labels:", test_class_labels)
```

Training class labels: {'angry': 0, 'disgust': 1, 'fear': 2, 'happy': 3, 'neutral': 4, 'sad': 5, 'surprise': 6}

Validation class labels: {'angry': 0, 'disgust': 1, 'fear': 2, 'happy': 3, 'neutral': 4, 'sad': 5, 'surprise': 6}

Validation class labels: {'angry': 0, 'disgust': 1, 'fear': 2, 'happy': 3, 'neutral': 4, 'sad': 5, 'surprise': 6}

```
In [22]: # Initialising the CNN
model = Sequential()

# Adding convolutional layers with activations on the same line for clarity
model.add(Conv2D(32, kernel_size=(3, 3), kernel_initializer="glorot_uniform", padding='same'))
model.add(Activation('relu'))
model.add(Conv2D(64, kernel_size=(3, 3), padding='same'))
model.add(Activation('relu'))
model.add(BatchNormalization())
model.add(MaxPooling2D(2, 2))
model.add(Dropout(0.25))

model.add(Conv2D(128, kernel_size=(3, 3), padding='same', kernel_regularizer=regularizers.l2(0.01)))
model.add(Activation('relu'))
model.add(Conv2D(256, kernel_size=(3, 3), kernel_regularizer=regularizers.l2(0.01)))
model.add(Activation('relu'))
model.add(BatchNormalization())
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Dropout(0.25))

model.add(Conv2D(512, kernel_size=(3, 3), padding='same', kernel_regularizer=regularizers.l2(0.01)))
model.add(Activation('relu'))
model.add(Conv2D(512, kernel_size=(3, 3), padding='same', kernel_regularizer=regularizers.l2(0.01)))
model.add(Activation('relu'))
model.add(BatchNormalization())
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Dropout(0.25))

# Flattening and adding dense layers
model.add(Flatten())
model.add(Dense(1024))
model.add(Activation('relu'))
model.add(Dropout(0.5))
```

```
# Output Layer  
model.add(Dense(num_classes))  
model.add(Activation('softmax'))
```

```
In [23]: model.summary()
```


Model: "sequential"

Layer (type)	Output Shape	Param #
=====		
conv2d (Conv2D)	(None, 48, 48, 32)	320
activation (Activation)	(None, 48, 48, 32)	0
conv2d_1 (Conv2D)	(None, 48, 48, 64)	18496
activation_1 (Activation)	(None, 48, 48, 64)	0
batch_normalization (Batch Normalization)	(None, 48, 48, 64)	256
max_pooling2d (MaxPooling2D)	(None, 24, 24, 64)	0
dropout (Dropout)	(None, 24, 24, 64)	0
conv2d_2 (Conv2D)	(None, 24, 24, 128)	73856
activation_2 (Activation)	(None, 24, 24, 128)	0
conv2d_3 (Conv2D)	(None, 22, 22, 256)	295168
activation_3 (Activation)	(None, 22, 22, 256)	0
batch_normalization_1 (Batch Normalization)	(None, 22, 22, 256)	1024
max_pooling2d_1 (MaxPooling2D)	(None, 11, 11, 256)	0
dropout_1 (Dropout)	(None, 11, 11, 256)	0
conv2d_4 (Conv2D)	(None, 11, 11, 512)	1180160
activation_4 (Activation)	(None, 11, 11, 512)	0
conv2d_5 (Conv2D)	(None, 11, 11, 512)	2359808
activation_5 (Activation)	(None, 11, 11, 512)	0
batch_normalization_2 (Batch Normalization)	(None, 11, 11, 512)	2048
max_pooling2d_2 (MaxPooling2D)	(None, 5, 5, 512)	0
dropout_2 (Dropout)	(None, 5, 5, 512)	0
flatten (Flatten)	(None, 12800)	0
dense (Dense)	(None, 1024)	13108224

activation_6 (Activation)	(None, 1024)	0
dropout_3 (Dropout)	(None, 1024)	0
dense_1 (Dense)	(None, 7)	7175
activation_7 (Activation)	(None, 7)	0

=====
Total params: 17046535 (65.03 MB)
Trainable params: 17044871 (65.02 MB)
Non-trainable params: 1664 (6.50 KB)

```
In [24]: # Compiling the model
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.0001),
              loss='categorical_crossentropy',
              metrics=['accuracy'])
```

```
In [25]: model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
=====		
conv2d (Conv2D)	(None, 48, 48, 32)	320
activation (Activation)	(None, 48, 48, 32)	0
conv2d_1 (Conv2D)	(None, 48, 48, 64)	18496
activation_1 (Activation)	(None, 48, 48, 64)	0
batch_normalization (Batch Normalization)	(None, 48, 48, 64)	256
max_pooling2d (MaxPooling2D)	(None, 24, 24, 64)	0
dropout (Dropout)	(None, 24, 24, 64)	0
conv2d_2 (Conv2D)	(None, 24, 24, 128)	73856
activation_2 (Activation)	(None, 24, 24, 128)	0
conv2d_3 (Conv2D)	(None, 22, 22, 256)	295168
activation_3 (Activation)	(None, 22, 22, 256)	0
batch_normalization_1 (Batch Normalization)	(None, 22, 22, 256)	1024
max_pooling2d_1 (MaxPooling2D)	(None, 11, 11, 256)	0
dropout_1 (Dropout)	(None, 11, 11, 256)	0
conv2d_4 (Conv2D)	(None, 11, 11, 512)	1180160
activation_4 (Activation)	(None, 11, 11, 512)	0
conv2d_5 (Conv2D)	(None, 11, 11, 512)	2359808
activation_5 (Activation)	(None, 11, 11, 512)	0
batch_normalization_2 (Batch Normalization)	(None, 11, 11, 512)	2048
max_pooling2d_2 (MaxPooling2D)	(None, 5, 5, 512)	0
dropout_2 (Dropout)	(None, 5, 5, 512)	0
flatten (Flatten)	(None, 12800)	0
dense (Dense)	(None, 1024)	13108224

activation_6 (Activation)	(None, 1024)	0
dropout_3 (Dropout)	(None, 1024)	0
dense_1 (Dense)	(None, 7)	7175
activation_7 (Activation)	(None, 7)	0

=====
Total params: 17046535 (65.03 MB)
Trainable params: 17044871 (65.02 MB)
Non-trainable params: 1664 (6.50 KB)

```
In [26]: plot_model(model, to_file = '/content/FER_2013_Emotion_Detection/Custom_CNN_From_Sc
          show_shapes=True, show_layer_names=True)
```

Out[26]:

conv2d_input	input:	[(None, 48, 48, 1)]
InputLayer	output:	[(None, 48, 48, 1)]



conv2d	input:	(None, 48, 48, 1)
Conv2D	output:	(None, 48, 48, 32)



activation	input:	(None, 48, 48, 32)
Activation	output:	(None, 48, 48, 32)



conv2d_1	input:	(None, 48, 48, 32)
Conv2D	output:	(None, 48, 48, 64)



activation_1	input:	(None, 48, 48, 64)
Activation	output:	(None, 48, 48, 64)



batch_normalization	input:	(None, 48, 48, 64)
BatchNormalization	output:	(None, 48, 48, 64)



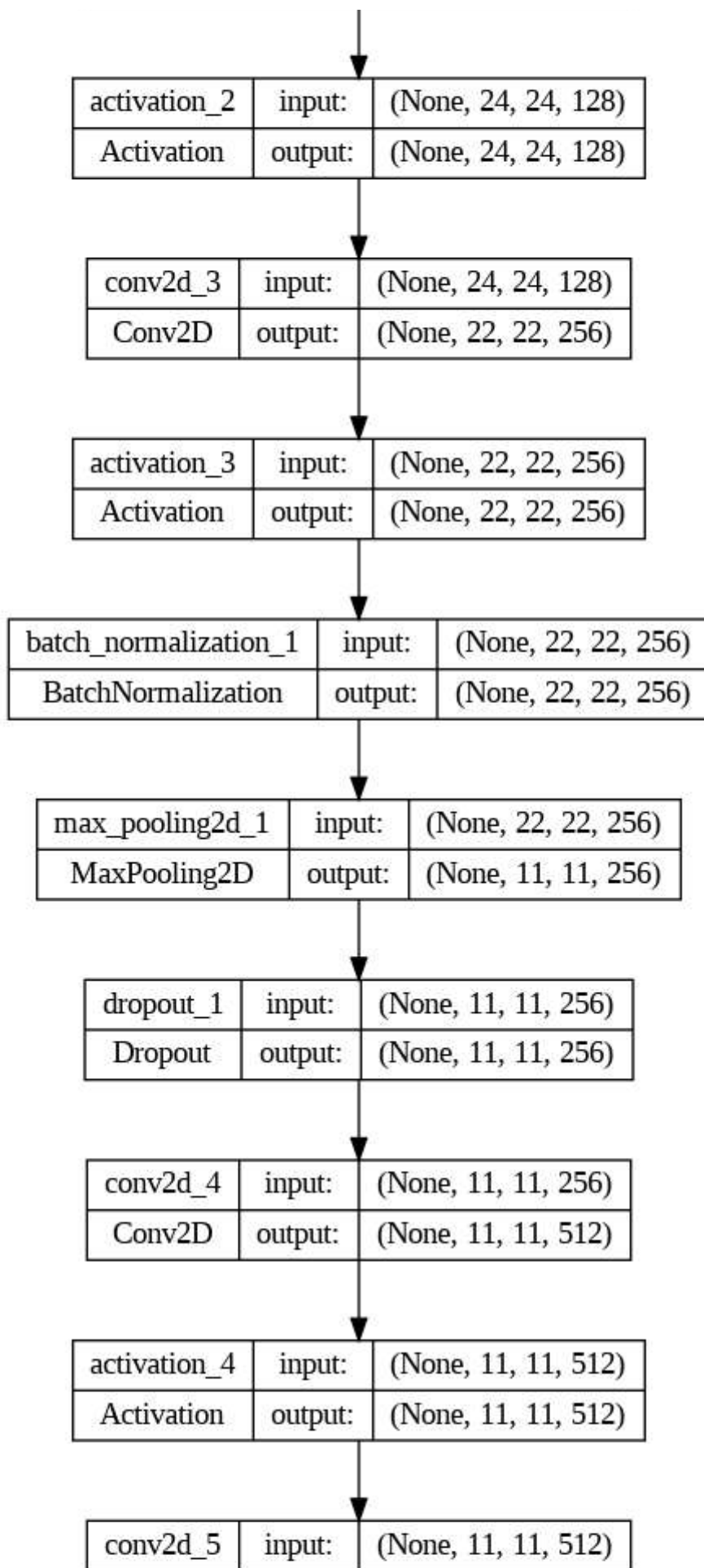
max_pooling2d	input:	(None, 48, 48, 64)
MaxPooling2D	output:	(None, 24, 24, 64)



dropout	input:	(None, 24, 24, 64)
Dropout	output:	(None, 24, 24, 64)



conv2d_2	input:	(None, 24, 24, 64)
Conv2D	output:	(None, 24, 24, 128)



Conv2D	output:	(None, 11, 11, 512)
--------	---------	---------------------

Understanding Callbacks

```
In [29]: # File path for the model checkpoint
cnn_path = '/content/FER_2013_Emotion_Detection/Custom_CNN_From_Scratch'
name = 'Custom_CNN_model.keras'
chk_path = os.path.join(cnn_path, name)
chk_path
```

```
Out[29]: '/content/FER_2013_Emotion_Detection/Custom_CNN_From_Scratch/Custom_CNN_model.keras'
```

BatchNormalization	output:	(None, 11, 11, 512)
--------------------	---------	---------------------

```
In [30]: # Callback to save the model checkpoint
checkpoint = ModelCheckpoint(filepath=chk_path,
                             save_best_only=True,
                             verbose=1,
```

max_pooling2d_2	input:	(None, 11, 11, 512)
MaxPooling2D	output:	(None, 5, 5, 512)

```
In [31]: # Callback for early stopping
earlystop = EarlyStopping(monitor='val_loss',
                           min_delta=0,
                           patience=10,
                           verbose=1,
                           restore_best_weights=True)
```

dropout_2	input:	(None, 5, 5, 512)
Dropout	output:	(None, 5, 5, 512)

```
In [32]: # Callback to reduce Learning rate
reduce_lr = ReduceLROnPlateau(monitor='val_loss',
                                factor=0.5,
                                patience=6,
                                verbose=1,
                                min_delta=0.0001)
```

flatten	input:	(None, 5, 5, 512)
Flatten	output:	(None, 12800)

```
In [33]: # Callback to Log training data to a CSV file
csv_logger = CSVLogger(os.path.join(cnn_path, 'training.log'))
```

dense	input:	(None, 12800)
Dense	output:	(None, 1024)

```
In [34]: # Aggregating all callbacks into a list
callbacks = [checkpoint, earlystop, reduce_lr, csv_logger] # Adjusted as per your
```

Calculating Steps for epoch

Activation	output:	(None, 1024)
------------	---------	--------------

```
In [35]: train_generator.samples
```

```
Out[35]: 22968
```

```
In [36]: train_steps_per_epoch = train_generator.samples // train_generator.batch_size + 1
validation_steps_epoch = validation_generator.samples // validation_generator.batch_size + 1
test_steps_epoch = test_generator.samples // test_generator.batch_size + 1
```

Model Training

dense_1	input:	(None, 1024)
Dense	output:	(None, 7)

```
In [37]: history = model.fit(  
    train_generator,  
    steps_per_epoch=train_steps_per_epoch,  
    epochs=10,  
    validation_data=validation_generator,  
    validation_steps=validation_steps_epoch,  
    callbacks=callbacks)
```

activation_7	input:	(None, 7)
Activation	output:	(None, 7)

Epoch 1/10
359/359 [=====] - ETA: 0s - loss: 12.4396 - accuracy: 0.2740
Epoch 1: val_loss improved from inf to 13.51567, saving model to /content/FER_2013_Emotion_Detection/Custom_CNN_From_Scratch/Custom_CNN_model.keras
359/359 [=====] - 40s 81ms/step - loss: 12.4396 - accuracy: 0.2740 - val_loss: 13.5157 - val_accuracy: 0.1735 - lr: 1.0000e-04
Epoch 2/10
359/359 [=====] - ETA: 0s - loss: 10.2490 - accuracy: 0.3507
Epoch 2: val_loss improved from 13.51567 to 9.49665, saving model to /content/FER_2013_Emotion_Detection/Custom_CNN_From_Scratch/Custom_CNN_model.keras
359/359 [=====] - 26s 71ms/step - loss: 10.2490 - accuracy: 0.3507 - val_loss: 9.4966 - val_accuracy: 0.3904 - lr: 1.0000e-04
Epoch 3/10
359/359 [=====] - ETA: 0s - loss: 8.2320 - accuracy: 0.3958
Epoch 3: val_loss improved from 9.49665 to 7.67904, saving model to /content/FER_2013_Emotion_Detection/Custom_CNN_From_Scratch/Custom_CNN_model.keras
359/359 [=====] - 25s 71ms/step - loss: 8.2320 - accuracy: 0.3958 - val_loss: 7.6790 - val_accuracy: 0.4059 - lr: 1.0000e-04
Epoch 4/10
359/359 [=====] - ETA: 0s - loss: 6.4891 - accuracy: 0.4364
Epoch 4: val_loss improved from 7.67904 to 5.75533, saving model to /content/FER_2013_Emotion_Detection/Custom_CNN_From_Scratch/Custom_CNN_model.keras
359/359 [=====] - 29s 81ms/step - loss: 6.4891 - accuracy: 0.4364 - val_loss: 5.7553 - val_accuracy: 0.4663 - lr: 1.0000e-04
Epoch 5/10
359/359 [=====] - ETA: 0s - loss: 5.1047 - accuracy: 0.4679
Epoch 5: val_loss improved from 5.75533 to 4.59007, saving model to /content/FER_2013_Emotion_Detection/Custom_CNN_From_Scratch/Custom_CNN_model.keras
359/359 [=====] - 27s 74ms/step - loss: 5.1047 - accuracy: 0.4679 - val_loss: 4.5901 - val_accuracy: 0.4825 - lr: 1.0000e-04
Epoch 6/10
359/359 [=====] - ETA: 0s - loss: 4.0593 - accuracy: 0.4923
Epoch 6: val_loss improved from 4.59007 to 3.72435, saving model to /content/FER_2013_Emotion_Detection/Custom_CNN_From_Scratch/Custom_CNN_model.keras
359/359 [=====] - 26s 72ms/step - loss: 4.0593 - accuracy: 0.4923 - val_loss: 3.7244 - val_accuracy: 0.4895 - lr: 1.0000e-04
Epoch 7/10
359/359 [=====] - ETA: 0s - loss: 3.2763 - accuracy: 0.5223
Epoch 7: val_loss improved from 3.72435 to 2.98442, saving model to /content/FER_2013_Emotion_Detection/Custom_CNN_From_Scratch/Custom_CNN_model.keras
359/359 [=====] - 26s 72ms/step - loss: 3.2763 - accuracy: 0.5223 - val_loss: 2.9844 - val_accuracy: 0.5262 - lr: 1.0000e-04
Epoch 8/10
359/359 [=====] - ETA: 0s - loss: 2.7262 - accuracy: 0.5433
Epoch 8: val_loss improved from 2.98442 to 2.50877, saving model to /content/FER_2013_Emotion_Detection/Custom_CNN_From_Scratch/Custom_CNN_model.keras
359/359 [=====] - 26s 72ms/step - loss: 2.7262 - accuracy: 0.5433 - val_loss: 2.5088 - val_accuracy: 0.5511 - lr: 1.0000e-04
Epoch 9/10
359/359 [=====] - ETA: 0s - loss: 2.3101 - accuracy: 0.5706
Epoch 9: val_loss improved from 2.50877 to 2.19617, saving model to /content/FER_2013_Emotion_Detection/Custom_CNN_From_Scratch/Custom_CNN_model.keras
359/359 [=====] - 26s 72ms/step - loss: 2.3101 - accuracy: 0.5706 - val_loss: 2.1962 - val_accuracy: 0.5487 - lr: 1.0000e-04

```
Epoch 10/10
359/359 [=====] - ETA: 0s - loss: 2.0176 - accuracy: 0.5856
Epoch 10: val_loss improved from 2.19617 to 2.01175, saving model to /content/FER_20
13_Emotion_Detection/Custom_CNN_From_Scratch/Custom_CNN_model.keras
359/359 [=====] - 26s 72ms/step - loss: 2.0176 - accuracy:
0.5856 - val_loss: 2.0118 - val_accuracy: 0.5469 - lr: 1.0000e-04
```

Plotting Performance Metrics

```
In [38]: def plot_training_history(history):
        """
        Plots the training and validation accuracy and loss.

        Parameters:
        - history: A Keras History object. Contains the logs from the training process.

        Returns:
        - None. Displays the matplotlib plots for training/validation accuracy and loss
        """
        acc = history.history['accuracy']
        val_acc = history.history['val_accuracy']
        loss = history.history['loss']
        val_loss = history.history['val_loss']

        epochs_range = range(len(acc))

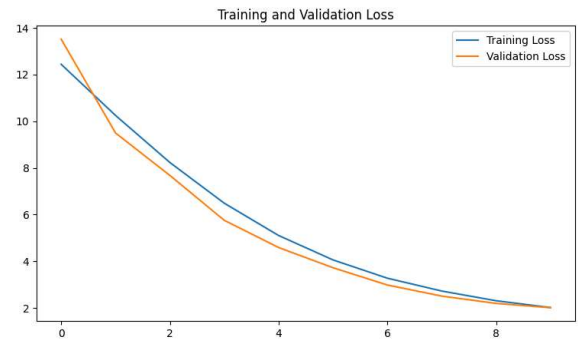
        plt.figure(figsize=(20, 5))

        # Plot training and validation accuracy
        plt.subplot(1, 2, 1)
        plt.plot(epochs_range, acc, label='Training Accuracy')
        plt.plot(epochs_range, val_acc, label='Validation Accuracy')
        plt.legend(loc='lower right')
        plt.title('Training and Validation Accuracy')

        # Plot training and validation loss
        plt.subplot(1, 2, 2)
        plt.plot(epochs_range, loss, label='Training Loss')
        plt.plot(epochs_range, val_loss, label='Validation Loss')
        plt.legend(loc='upper right')
        plt.title('Training and Validation Loss')

        plt.show()
```

```
In [39]: plot_training_history(history)
```



Model Evaluation

```
In [ ]: train_loss, train_accu = model.evaluate(train_generator)
        test_loss, test_accu = model.evaluate(test_generator)
        print("final train accuracy = {:.2f} , validation accuracy = {:.2f}".format(train_a

359/359 [=====] - 10s 28ms/step - loss: 1.8045 - accuracy:
0.6311
113/113 [=====] - 3s 27ms/step - loss: 2.0106 - accuracy:
0.5552
final train accuracy = 63.11 , validation accuracy = 55.52
```

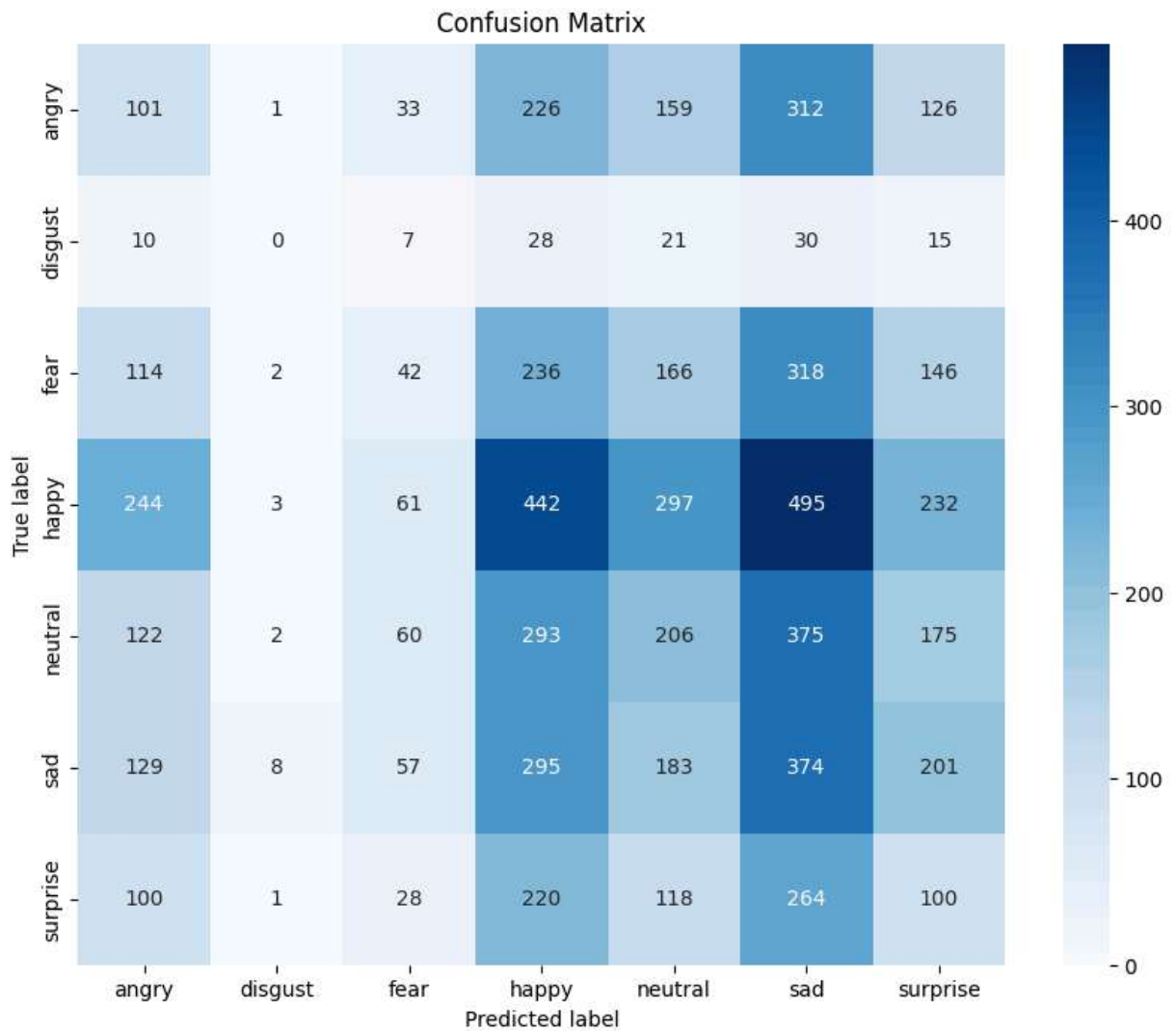
Plotting Confusion Matrix

```
In [40]: # Assuming your true_classes and predicted_classes are already defined
true_classes = test_generator.classes
predicted_classes = np.argmax(model.predict(test_generator, steps=np.ceil(test_gene
class_labels = list(test_generator.class_indices.keys())

# Generate the confusion matrix
cm = confusion_matrix(true_classes, predicted_classes)

# Plotting with seaborn
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=class_labels, ytickl
plt.title('Confusion Matrix')
plt.ylabel('True label')
plt.xlabel('Predicted label')
plt.show()

113/113 [=====] - 4s 35ms/step
```



```
In [41]: # Printing the classification report
report = classification_report(true_classes, predicted_classes, target_names=class_
print("Classification Report:\n", report)
```

Classification Report:

	precision	recall	f1-score	support
angry	0.12	0.11	0.11	958
disgust	0.00	0.00	0.00	111
fear	0.15	0.04	0.06	1024
happy	0.25	0.25	0.25	1774
neutral	0.18	0.17	0.17	1233
sad	0.17	0.30	0.22	1247
surprise	0.10	0.12	0.11	831
accuracy			0.18	7178
macro avg	0.14	0.14	0.13	7178
weighted avg	0.17	0.18	0.17	7178

Making Predictions

```
In [43]: # Emotion classes for the dataset
Emotion_Classes = ['Angry', 'Disgust', 'Fear', 'Happy', 'Neutral', 'Sad', 'Surprise']

# Assuming test_generator and model are already defined
batch_size = test_generator.batch_size

# Selecting a random batch from the test generator
Random_batch = np.random.randint(0, len(test_generator) - 1)

# Selecting random image indices from the batch
Random_Img_Index = np.random.randint(0, batch_size, 10)

# Setting up the plot
fig, axes = plt.subplots(nrows=2, ncols=5, figsize=(10, 5),
                        subplot_kw={'xticks': [], 'yticks': []})

for i, ax in enumerate(axes.flat):
    # Fetching the random image and its label
    Random_Img = test_generator[Random_batch][0][Random_Img_Index[i]]
    Random_Img_Label = np.argmax(test_generator[Random_batch][1][Random_Img_Index[i]])

    # Making a prediction using the model
    Model_Prediction = np.argmax(model.predict(tf.expand_dims(Random_Img, axis=0)),

    # Displaying the image
    ax.imshow(Random_Img.squeeze(), cmap='gray') # Assuming the images are grayscale
    # Setting the title with true and predicted labels, colored based on correctness
    color = "green" if Emotion_Classes[Random_Img_Label] == Emotion_Classes[Model_Prediction] else "red"
    ax.set_title(f"True: {Emotion_Classes[Random_Img_Label]}\nPredicted: {Emotion_Classes[Model_Prediction]}")

plt.tight_layout()
plt.show()
```



Model 2: Image Augmentation

```

In [44]: # Define paths to the train and validation directories
train_data_dir = '/content/train'
test_data_dir = '/content/test'
# validation_data_dir = '/content/emotion_detection_project/datasets/raw/test'

# Set some parameters
img_width, img_height = 48, 48 # Size of images
batch_size = 64
epochs = 10
num_classes = 7 # Update this based on the number of your classes

# Initializing the ImageDataGenerator with data augmentation options for the traini
data_generator = ImageDataGenerator(
    rescale=1./255, # Rescale the pixel values from [0, 255] to [0, 1]
    rotation_range=40, # Degree range for random rotations
    width_shift_range=0.2, # Range (as a fraction of total width) for random horiz
    height_shift_range=0.2, # Range (as a fraction of total height) for random ver
    shear_range=0.2, # Shearing intensity (shear angle in counter-clockwise direct
    zoom_range=0.2, # Range for random zoom
    horizontal_flip=True, # Randomly flip inputs horizontally
    fill_mode='nearest', # Strategy to fill newly created pixels, which can appear
    validation_split=0.2 # Set the validation split; 20% of the data will be used
)

test_data_generator = ImageDataGenerator(rescale=1./255)

# Automatically retrieve images and their classes for train and validation sets
train_generator = data_generator.flow_from_directory(
    train_data_dir,
    target_size=(img_width, img_height),
    batch_size=batch_size,
    class_mode='categorical',
    color_mode='grayscale',
    subset='training')

validation_generator = data_generator.flow_from_directory(
    train_data_dir,
    target_size=(img_width, img_height),
    batch_size=batch_size,
    class_mode='categorical',
    color_mode='grayscale',
    subset='validation')

test_generator = test_data_generator.flow_from_directory(
    test_data_dir,
    target_size=(img_width, img_height),
    batch_size=batch_size,
    class_mode='categorical',
    color_mode='grayscale'
)

```

Found 22968 images belonging to 7 classes.

Found 5741 images belonging to 7 classes.

Found 7178 images belonging to 7 classes.

```
In [45]: # Path to your specific image
image_path = '/content/train/angry/Training_10118481.jpg'

# Load and convert the image to an array
img = load_img(image_path, color_mode='grayscale', target_size=(img_width, img_height))
img_array = img_to_array(img) # Converts the image to a numpy array
img_array = img_array.reshape((1,) + img_array.shape) # Reshapes the image to (1,

# Setting up the plot
fig, axes = plt.subplots(nrows=1, ncols=5, figsize=(10, 4))
# Plot the original image first. Since img_array is a 3D array after reshaping, we
axes[0].imshow(img_array[0, :, :, 0], cmap='gray')
axes[0].set_title('Original Image')
axes[0].axis('off')

# Generate and plot augmented images
for i, ax in enumerate(axes.flat[1:]): # Start from the second subplot
    # Generate a batch of augmented images
    aug_iter = data_generator.flow(img_array, batch_size=1)
    aug_img = next(aug_iter)[0] # Get the first augmented image from the batch

    # Plot the augmented image. We directly use[:, :, 0] without an initial batch
    ax.imshow(aug_img[:, :, 0], cmap='gray')
    ax.set_title(f'Augmented Image {i+1}')
    ax.axis('off')

plt.tight_layout()
plt.show()
```



Same CNN Model as above

```
In [50]: # Assuming img_width, img_height, and classes are defined. For example:
img_width, img_height = 48, 48
classes = 7

#Initialising the CNN
model = tf.keras.models.Sequential()

# Adding convolutional Layers with activations on separate Lines
model.add(Conv2D(32, kernel_size=(3, 3), padding='same', input_shape=(img_width, img_height, 1)))
model.add(tf.keras.layers.Activation('relu'))
```



```

model.add(Conv2D(64, kernel_size=(3, 3), padding='same'))
model.add(tf.keras.layers.Activation('relu'))
model.add(BatchNormalization())
model.add(MaxPooling2D(2, 2))
model.add(Dropout(0.25))

model.add(Conv2D(128, kernel_size=(3, 3), padding='same', kernel_regularizer=regularizers.l2(0.01)))
model.add(tf.keras.layers.Activation('relu'))
model.add(Conv2D(256, kernel_size=(3, 3), kernel_regularizer=regularizers.l2(0.01)))
model.add(tf.keras.layers.Activation('relu'))
model.add(BatchNormalization())
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Dropout(0.25))

model.add(Conv2D(512, kernel_size=(3, 3), padding='same', kernel_regularizer=regularizers.l2(0.01)))
model.add(tf.keras.layers.Activation('relu'))
model.add(Conv2D(512, kernel_size=(3, 3), padding='same', kernel_regularizer=regularizers.l2(0.01)))
model.add(tf.keras.layers.Activation('relu'))
model.add(BatchNormalization())
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Dropout(0.25))

# Flattening and adding dense layers
model.add(Flatten())
model.add(Dense(1024))
model.add(tf.keras.layers.Activation('relu'))
model.add(Dropout(0.5))

model.add(Dense(classes))
model.add(tf.keras.layers.Activation('softmax'))

# Compiling the model
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.0001),
              loss='categorical_crossentropy',
              metrics=['accuracy'])

# File path for the model checkpoint
cnn_path = '/content/FER_2013_Emotion_Detection/Custom_CNN_With_Augmentation'
name = 'Custom_CNN_augmented_model.keras'
chk_path = os.path.join(cnn_path, name)

# Callback to save the model checkpoint
checkpoint = ModelCheckpoint(filepath=chk_path,
                             save_best_only=True,
                             verbose=1,
                             mode='min',
                             monitor='val_loss')

# Callback for early stopping
earlystop = EarlyStopping(monitor='val_loss',
                          min_delta=0,
                          patience=3,
                          verbose=1,
                          restore_best_weights=True)

```



```

# Callback to reduce Learning rate
reduce_lr = ReduceLROnPlateau(monitor='val_loss',
                               factor=0.2,
                               patience=6,
                               verbose=1,
                               min_delta=0.0001)

# Callback to Log training data to a CSV file
csv_logger = CSVLogger(os.path.join(cnn_path, 'training.log'))

# Aggregating all callbacks into a list
callbacks = [checkpoint, earlystop, reduce_lr, csv_logger] # Adjusted as per your

train_steps_per_epoch = train_generator.samples // train_generator.batch_size + 1
validation_steps_epoch = validation_generator.samples // validation_generator.batch_size + 1
test_steps_epoch = test_generator.samples // test_generator.batch_size + 1

history = model.fit(
    train_generator,
    steps_per_epoch=train_steps_per_epoch,
    epochs=100,
    validation_data=validation_generator,
    validation_steps=validation_steps_epoch,
    callbacks=callbacks)

```

Epoch 1/100
359/359 [=====] - ETA: 0s - loss: 12.4896 - accuracy: 0.2188
Epoch 1: val_loss improved from inf to 14.14767, saving model to /content/FER_2013_Emotion_Detection/Custom_CNN_With_Augmentation/Custom_CNN_augmented_model.keras
359/359 [=====] - 37s 87ms/step - loss: 12.4896 - accuracy: 0.2188 - val_loss: 14.1477 - val_accuracy: 0.1681 - lr: 1.0000e-04
Epoch 2/100
359/359 [=====] - ETA: 0s - loss: 10.2419 - accuracy: 0.2406
Epoch 2: val_loss improved from 14.14767 to 10.43937, saving model to /content/FER_2013_Emotion_Detection/Custom_CNN_With_Augmentation/Custom_CNN_augmented_model.keras
359/359 [=====] - 32s 90ms/step - loss: 10.2419 - accuracy: 0.2406 - val_loss: 10.4394 - val_accuracy: 0.2284 - lr: 1.0000e-04
Epoch 3/100
359/359 [=====] - ETA: 0s - loss: 8.1725 - accuracy: 0.2461
Epoch 3: val_loss improved from 10.43937 to 7.88726, saving model to /content/FER_2013_Emotion_Detection/Custom_CNN_With_Augmentation/Custom_CNN_augmented_model.keras
359/359 [=====] - 31s 87ms/step - loss: 8.1725 - accuracy: 0.2461 - val_loss: 7.8873 - val_accuracy: 0.2439 - lr: 1.0000e-04
Epoch 4/100
359/359 [=====] - ETA: 0s - loss: 6.3900 - accuracy: 0.2496
Epoch 4: val_loss improved from 7.88726 to 6.55996, saving model to /content/FER_2013_Emotion_Detection/Custom_CNN_With_Augmentation/Custom_CNN_augmented_model.keras
359/359 [=====] - 32s 90ms/step - loss: 6.3900 - accuracy: 0.2496 - val_loss: 6.5600 - val_accuracy: 0.2555 - lr: 1.0000e-04
Epoch 5/100
359/359 [=====] - ETA: 0s - loss: 4.9852 - accuracy: 0.2593
Epoch 5: val_loss improved from 6.55996 to 4.91589, saving model to /content/FER_2013_Emotion_Detection/Custom_CNN_With_Augmentation/Custom_CNN_augmented_model.keras
359/359 [=====] - 32s 89ms/step - loss: 4.9852 - accuracy: 0.2593 - val_loss: 4.9159 - val_accuracy: 0.2595 - lr: 1.0000e-04
Epoch 6/100
359/359 [=====] - ETA: 0s - loss: 3.9425 - accuracy: 0.2672
Epoch 6: val_loss improved from 4.91589 to 3.85648, saving model to /content/FER_2013_Emotion_Detection/Custom_CNN_With_Augmentation/Custom_CNN_augmented_model.keras
359/359 [=====] - 32s 90ms/step - loss: 3.9425 - accuracy: 0.2672 - val_loss: 3.8565 - val_accuracy: 0.2555 - lr: 1.0000e-04
Epoch 7/100
359/359 [=====] - ETA: 0s - loss: 3.2123 - accuracy: 0.2734
Epoch 7: val_loss improved from 3.85648 to 2.92475, saving model to /content/FER_2013_Emotion_Detection/Custom_CNN_With_Augmentation/Custom_CNN_augmented_model.keras
359/359 [=====] - 31s 86ms/step - loss: 3.2123 - accuracy: 0.2734 - val_loss: 2.9248 - val_accuracy: 0.2757 - lr: 1.0000e-04
Epoch 8/100
359/359 [=====] - ETA: 0s - loss: 2.7132 - accuracy: 0.2847
Epoch 8: val_loss improved from 2.92475 to 2.50497, saving model to /content/FER_2013_Emotion_Detection/Custom_CNN_With_Augmentation/Custom_CNN_augmented_model.keras
359/359 [=====] - 33s 91ms/step - loss: 2.7132 - accuracy: 0.2847 - val_loss: 2.5050 - val_accuracy: 0.2958 - lr: 1.0000e-04
Epoch 9/100
359/359 [=====] - ETA: 0s - loss: 2.3783 - accuracy: 0.2999
Epoch 9: val_loss improved from 2.50497 to 2.25481, saving model to /content/FER_2013_Emotion_Detection/Custom_CNN_With_Augmentation/Custom_CNN_augmented_model.keras
359/359 [=====] - 32s 88ms/step - loss: 2.3783 - accuracy: 0.2999 - val_loss: 2.2548 - val_accuracy: 0.3076 - lr: 1.0000e-04

Epoch 10/100
359/359 [=====] - ETA: 0s - loss: 2.1473 - accuracy: 0.3241
Epoch 10: val_loss improved from 2.25481 to 2.18143, saving model to /content/FER_2013_Emotion_Detection/Custom_CNN_With_Augmentation/Custom_CNN_augmented_model.keras
359/359 [=====] - 32s 90ms/step - loss: 2.1473 - accuracy: 0.3241 - val_loss: 2.1814 - val_accuracy: 0.3059 - lr: 1.0000e-04
Epoch 11/100
359/359 [=====] - ETA: 0s - loss: 1.9723 - accuracy: 0.3582
Epoch 11: val_loss improved from 2.18143 to 2.13723, saving model to /content/FER_2013_Emotion_Detection/Custom_CNN_With_Augmentation/Custom_CNN_augmented_model.keras
359/359 [=====] - 33s 91ms/step - loss: 1.9723 - accuracy: 0.3582 - val_loss: 2.1372 - val_accuracy: 0.3278 - lr: 1.0000e-04
Epoch 12/100
359/359 [=====] - ETA: 0s - loss: 1.8481 - accuracy: 0.3914
Epoch 12: val_loss improved from 2.13723 to 1.82506, saving model to /content/FER_2013_Emotion_Detection/Custom_CNN_With_Augmentation/Custom_CNN_augmented_model.keras
359/359 [=====] - 32s 89ms/step - loss: 1.8481 - accuracy: 0.3914 - val_loss: 1.8251 - val_accuracy: 0.3909 - lr: 1.0000e-04
Epoch 13/100
359/359 [=====] - ETA: 0s - loss: 1.7474 - accuracy: 0.4194
Epoch 13: val_loss improved from 1.82506 to 1.68063, saving model to /content/FER_2013_Emotion_Detection/Custom_CNN_With_Augmentation/Custom_CNN_augmented_model.keras
359/359 [=====] - 33s 91ms/step - loss: 1.7474 - accuracy: 0.4194 - val_loss: 1.6806 - val_accuracy: 0.4470 - lr: 1.0000e-04
Epoch 14/100
359/359 [=====] - ETA: 0s - loss: 1.6654 - accuracy: 0.4458
Epoch 14: val_loss did not improve from 1.68063
359/359 [=====] - 31s 86ms/step - loss: 1.6654 - accuracy: 0.4458 - val_loss: 1.8633 - val_accuracy: 0.4029 - lr: 1.0000e-04
Epoch 15/100
359/359 [=====] - ETA: 0s - loss: 1.6095 - accuracy: 0.4634
Epoch 15: val_loss improved from 1.68063 to 1.65176, saving model to /content/FER_2013_Emotion_Detection/Custom_CNN_With_Augmentation/Custom_CNN_augmented_model.keras
359/359 [=====] - 32s 89ms/step - loss: 1.6095 - accuracy: 0.4634 - val_loss: 1.6518 - val_accuracy: 0.4450 - lr: 1.0000e-04
Epoch 16/100
359/359 [=====] - ETA: 0s - loss: 1.5614 - accuracy: 0.4794
Epoch 16: val_loss improved from 1.65176 to 1.53894, saving model to /content/FER_2013_Emotion_Detection/Custom_CNN_With_Augmentation/Custom_CNN_augmented_model.keras
359/359 [=====] - 32s 90ms/step - loss: 1.5614 - accuracy: 0.4794 - val_loss: 1.5389 - val_accuracy: 0.4701 - lr: 1.0000e-04
Epoch 17/100
359/359 [=====] - ETA: 0s - loss: 1.5181 - accuracy: 0.4958
Epoch 17: val_loss improved from 1.53894 to 1.47200, saving model to /content/FER_2013_Emotion_Detection/Custom_CNN_With_Augmentation/Custom_CNN_augmented_model.keras
359/359 [=====] - 37s 103ms/step - loss: 1.5181 - accuracy: 0.4958 - val_loss: 1.4720 - val_accuracy: 0.5125 - lr: 1.0000e-04
Epoch 18/100
359/359 [=====] - ETA: 0s - loss: 1.4863 - accuracy: 0.5088
Epoch 18: val_loss did not improve from 1.47200
359/359 [=====] - 31s 86ms/step - loss: 1.4863 - accuracy: 0.5088 - val_loss: 1.4809 - val_accuracy: 0.5018 - lr: 1.0000e-04
Epoch 19/100
359/359 [=====] - ETA: 0s - loss: 1.4665 - accuracy: 0.5090
Epoch 19: val_loss improved from 1.47200 to 1.44207, saving model to /content/FER_2013_Emotion_Detection/Custom_CNN_With_Augmentation/Custom_CNN_augmented_model.keras

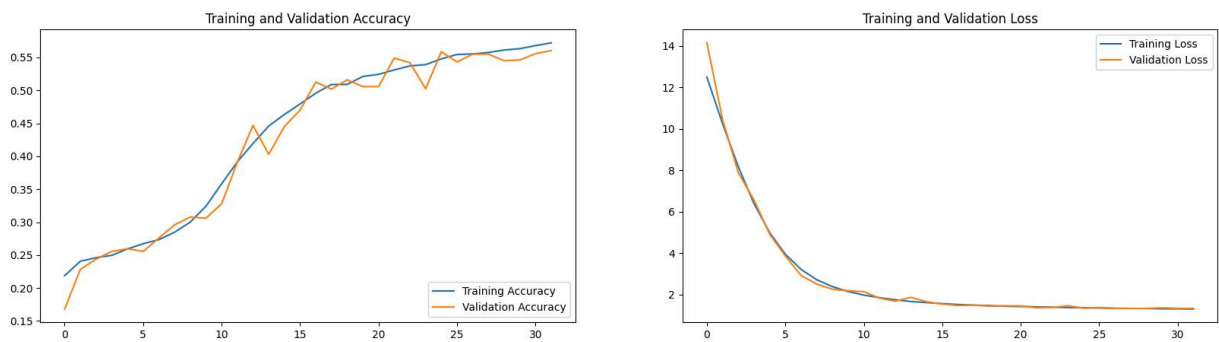
359/359 [=====] - 33s 91ms/step - loss: 1.4665 - accuracy: 0.5090 - val_loss: 1.4421 - val_accuracy: 0.5159 - lr: 1.0000e-04
Epoch 20/100
359/359 [=====] - ETA: 0s - loss: 1.4327 - accuracy: 0.5211
Epoch 20: val_loss did not improve from 1.44207
359/359 [=====] - 32s 88ms/step - loss: 1.4327 - accuracy: 0.5211 - val_loss: 1.4498 - val_accuracy: 0.5057 - lr: 1.0000e-04
Epoch 21/100
359/359 [=====] - ETA: 0s - loss: 1.4209 - accuracy: 0.5243
Epoch 21: val_loss did not improve from 1.44207
359/359 [=====] - 31s 86ms/step - loss: 1.4209 - accuracy: 0.5243 - val_loss: 1.4487 - val_accuracy: 0.5057 - lr: 1.0000e-04
Epoch 22/100
359/359 [=====] - ETA: 0s - loss: 1.4031 - accuracy: 0.5310
Epoch 22: val_loss improved from 1.44207 to 1.35939, saving model to /content/FER_2013_Emotion_Detection/Custom_CNN_With_Augmentation/Custom_CNN_augmented_model.keras
359/359 [=====] - 37s 104ms/step - loss: 1.4031 - accuracy: 0.5310 - val_loss: 1.3594 - val_accuracy: 0.5490 - lr: 1.0000e-04
Epoch 23/100
359/359 [=====] - ETA: 0s - loss: 1.3878 - accuracy: 0.5371
Epoch 23: val_loss did not improve from 1.35939
359/359 [=====] - 32s 88ms/step - loss: 1.3878 - accuracy: 0.5371 - val_loss: 1.3745 - val_accuracy: 0.5419 - lr: 1.0000e-04
Epoch 24/100
359/359 [=====] - ETA: 0s - loss: 1.3723 - accuracy: 0.5391
Epoch 24: val_loss did not improve from 1.35939
359/359 [=====] - 32s 88ms/step - loss: 1.3723 - accuracy: 0.5391 - val_loss: 1.4544 - val_accuracy: 0.5024 - lr: 1.0000e-04
Epoch 25/100
359/359 [=====] - ETA: 0s - loss: 1.3634 - accuracy: 0.5478
Epoch 25: val_loss improved from 1.35939 to 1.34036, saving model to /content/FER_2013_Emotion_Detection/Custom_CNN_With_Augmentation/Custom_CNN_augmented_model.keras
359/359 [=====] - 37s 102ms/step - loss: 1.3634 - accuracy: 0.5478 - val_loss: 1.3404 - val_accuracy: 0.5586 - lr: 1.0000e-04
Epoch 26/100
359/359 [=====] - ETA: 0s - loss: 1.3479 - accuracy: 0.5545
Epoch 26: val_loss did not improve from 1.34036
359/359 [=====] - 31s 87ms/step - loss: 1.3479 - accuracy: 0.5545 - val_loss: 1.3702 - val_accuracy: 0.5431 - lr: 1.0000e-04
Epoch 27/100
359/359 [=====] - ETA: 0s - loss: 1.3354 - accuracy: 0.5553
Epoch 27: val_loss improved from 1.34036 to 1.32841, saving model to /content/FER_2013_Emotion_Detection/Custom_CNN_With_Augmentation/Custom_CNN_augmented_model.keras
359/359 [=====] - 42s 117ms/step - loss: 1.3354 - accuracy: 0.5553 - val_loss: 1.3284 - val_accuracy: 0.5550 - lr: 1.0000e-04
Epoch 28/100
359/359 [=====] - ETA: 0s - loss: 1.3285 - accuracy: 0.5575
Epoch 28: val_loss did not improve from 1.32841
359/359 [=====] - 32s 88ms/step - loss: 1.3285 - accuracy: 0.5575 - val_loss: 1.3288 - val_accuracy: 0.5550 - lr: 1.0000e-04
Epoch 29/100
359/359 [=====] - ETA: 0s - loss: 1.3249 - accuracy: 0.5613
Epoch 29: val_loss improved from 1.32841 to 1.32682, saving model to /content/FER_2013_Emotion_Detection/Custom_CNN_With_Augmentation/Custom_CNN_augmented_model.keras
359/359 [=====] - 33s 90ms/step - loss: 1.3249 - accuracy: 0.5613 - val_loss: 1.3268 - val_accuracy: 0.5450 - lr: 1.0000e-04

```

Epoch 30/100
359/359 [=====] - ETA: 0s - loss: 1.3101 - accuracy: 0.5633
Epoch 30: val_loss did not improve from 1.32682
359/359 [=====] - 33s 91ms/step - loss: 1.3101 - accuracy:
0.5633 - val_loss: 1.3572 - val_accuracy: 0.5461 - lr: 1.0000e-04
Epoch 31/100
359/359 [=====] - ETA: 0s - loss: 1.3071 - accuracy: 0.5679
Epoch 31: val_loss did not improve from 1.32682
359/359 [=====] - 31s 86ms/step - loss: 1.3071 - accuracy:
0.5679 - val_loss: 1.3299 - val_accuracy: 0.5557 - lr: 1.0000e-04
Epoch 32/100
359/359 [=====] - ETA: 0s - loss: 1.2990 - accuracy: 0.5721
Epoch 32: val_loss did not improve from 1.32682
Restoring model weights from the end of the best epoch: 29.
359/359 [=====] - 31s 87ms/step - loss: 1.2990 - accuracy:
0.5721 - val_loss: 1.3316 - val_accuracy: 0.5604 - lr: 1.0000e-04
Epoch 32: early stopping

```

In [51]: `plot_training_history(history)`



In [52]: `train_loss, train_accu = model.evaluate(train_generator)`
`test_loss, test_accu = model.evaluate(test_generator)`
`print("final train accuracy = {:.2f} , validation accuracy = {:.2f}".format(train_a`

```

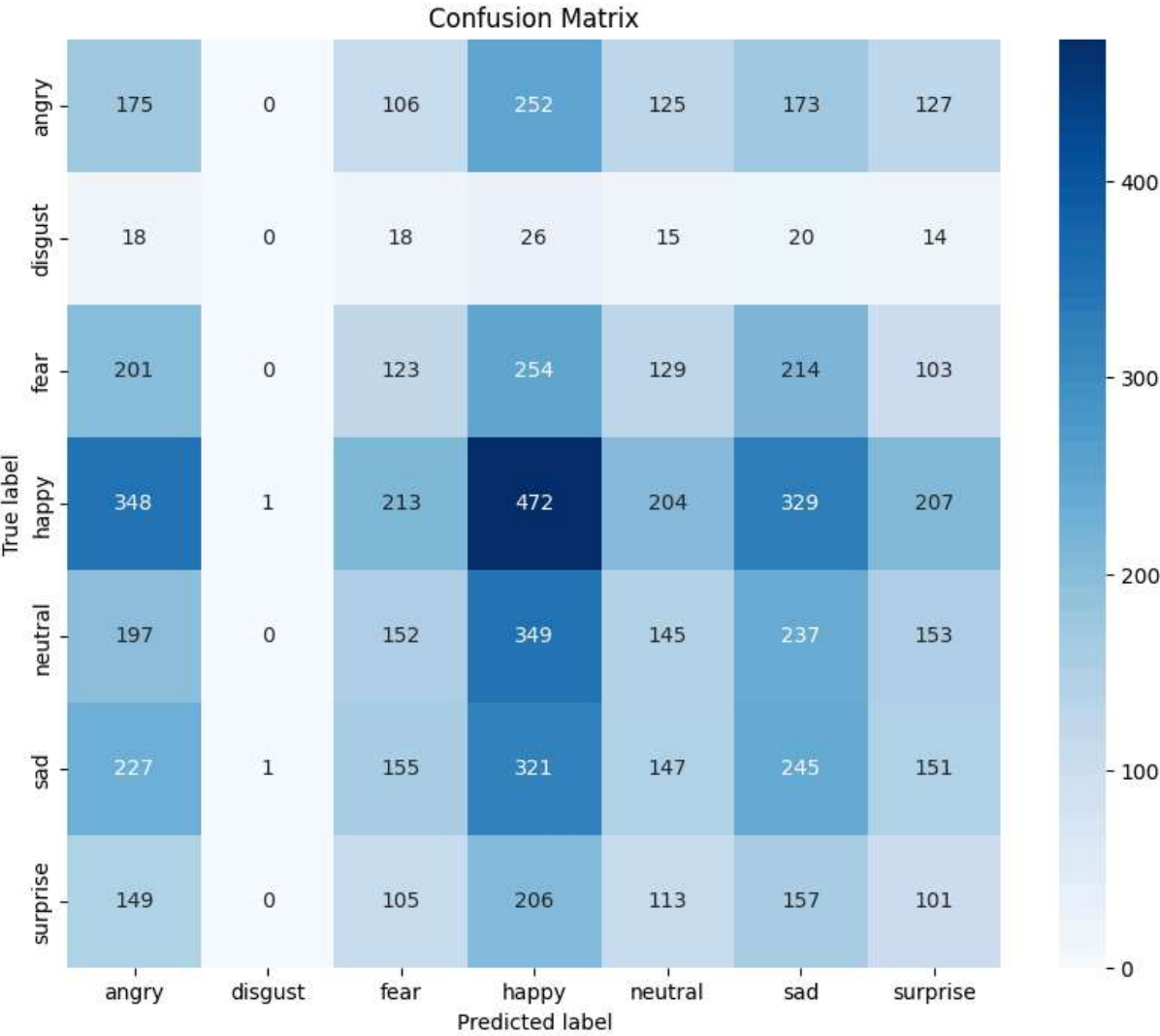
359/359 [=====] - 21s 58ms/step - loss: 1.2739 - accuracy:
0.5764
113/113 [=====] - 3s 24ms/step - loss: 1.2292 - accuracy:
0.5928
final train accuracy = 57.64 , validation accuracy = 59.28

```

In [53]: `# Assuming your true_classes and predicted_classes are already defined`
`true_classes = test_generator.classes`
`predicted_classes = np.argmax(model.predict(test_generator, steps=np.ceil(test_gene`
`class_labels = list(test_generator.class_indices.keys())`

`# Generate the confusion matrix`
`cm = confusion_matrix(true_classes, predicted_classes)`

`# Plotting with seaborn`
`plt.figure(figsize=(10, 8))`
`sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=class_labels, ytickl`
`plt.title('Confusion Matrix')`
`plt.ylabel('True label')`
`plt.xlabel('Predicted label')`
`plt.show()`

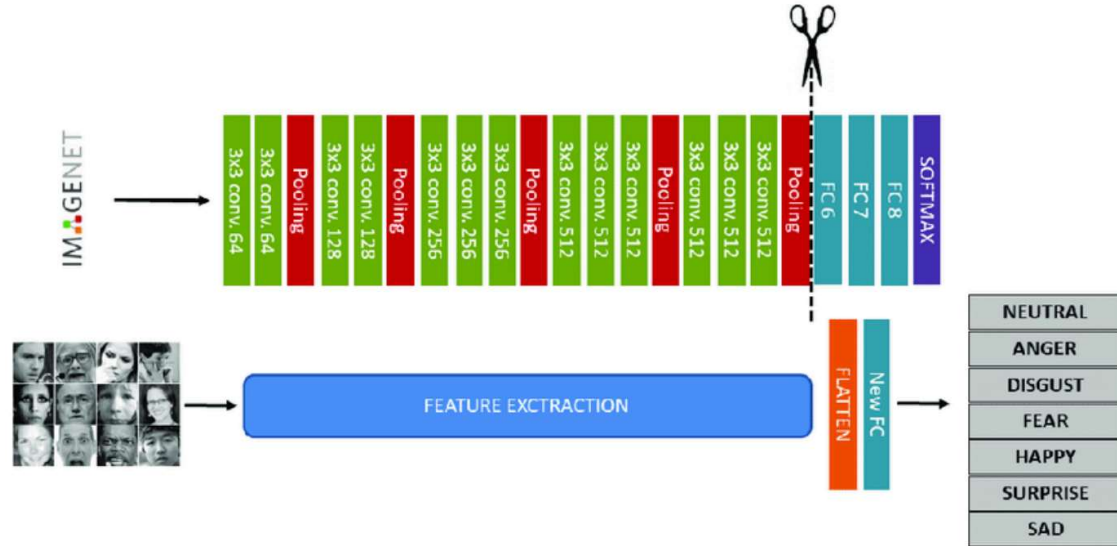


```
In [ ]: # Printing the classification report
report = classification_report(true_classes,
                               predicted_classes,
                               target_names=class_labels,
                               zero_division=0)
print("Classification Report:\n", report)
```

Classification Report:

	precision	recall	f1-score	support
angry	0.14	0.04	0.06	958
disgust	0.00	0.00	0.00	111
fear	0.14	0.03	0.05	1024
happy	0.25	0.23	0.24	1774
neutral	0.17	0.23	0.19	1233
sad	0.18	0.35	0.24	1247
surprise	0.13	0.13	0.13	831
accuracy			0.18	7178
macro avg	0.14	0.15	0.13	7178
weighted avg	0.17	0.18	0.16	7178

Model 3: Transfer Learning VGGNET



```
In [ ]: # Define paths to the train and validation directories
train_data_dir = '/content/train'
test_data_dir = '/content/test'

# validation_data_dir = '/content/emotion_detection_project/datasets/raw/test'

# Set some parameters
img_width, img_height = 224, 224 # Size of images
batch_size = 64
epochs = 10
num_classes = 7 # Update this based on the number of your classes

# Rescale the pixel values (0-255) to the [0, 1] interval
data_generator = ImageDataGenerator(
    rescale = 1 / 255.,
    rotation_range=10,
    zoom_range=0.2,
    width_shift_range=0.1,
    height_shift_range=0.1,
    horizontal_flip=True,
    fill_mode='nearest',
    # validation_split=0.2
)

test_preprocessor = ImageDataGenerator(
    rescale = 1 / 255.,
)

# Automatically retrieve images and their classes for train and validation sets
train_generator = data_generator.flow_from_directory(
    train_data_dir,
    target_size=(img_width, img_height),
```

```

        batch_size=batch_size,
        class_mode='categorical',
        color_mode='rgb',
        subset='training',
        shuffle = True)

test_generator = test_preprocessor.flow_from_directory(
    test_data_dir,
    target_size=(img_width, img_height),
    batch_size=batch_size,
    class_mode='categorical',
    color_mode='rgb',)
    # subset='validation')

```

Found 28709 images belonging to 7 classes.

Found 7178 images belonging to 7 classes.

```

In [ ]: classes = 7

# Clear the previous TensorFlow sessionx
tf.keras.backend.clear_session()

# Load the VGG16 base model, excluding its top (fully connected) layers
vgg = VGG16(input_shape=(224, 224, 3), include_top=False, weights='imagenet')
vgg.summary()

```


Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/vgg16/vgg16_weights_tf_dim_ordering_tf_kernels_notop.h5
58889256/58889256 [=====] - 3s 0us/step
Model: "vgg16"

Layer (type)	Output Shape	Param #
=====		
input_1 (InputLayer)	[(None, 224, 224, 3)]	0
block1_conv1 (Conv2D)	(None, 224, 224, 64)	1792
block1_conv2 (Conv2D)	(None, 224, 224, 64)	36928
block1_pool (MaxPooling2D)	(None, 112, 112, 64)	0
block2_conv1 (Conv2D)	(None, 112, 112, 128)	73856
block2_conv2 (Conv2D)	(None, 112, 112, 128)	147584
block2_pool (MaxPooling2D)	(None, 56, 56, 128)	0
block3_conv1 (Conv2D)	(None, 56, 56, 256)	295168
block3_conv2 (Conv2D)	(None, 56, 56, 256)	590080
block3_conv3 (Conv2D)	(None, 56, 56, 256)	590080
block3_pool (MaxPooling2D)	(None, 28, 28, 256)	0
block4_conv1 (Conv2D)	(None, 28, 28, 512)	1180160
block4_conv2 (Conv2D)	(None, 28, 28, 512)	2359808
block4_conv3 (Conv2D)	(None, 28, 28, 512)	2359808
block4_pool (MaxPooling2D)	(None, 14, 14, 512)	0
block5_conv1 (Conv2D)	(None, 14, 14, 512)	2359808
block5_conv2 (Conv2D)	(None, 14, 14, 512)	2359808
block5_conv3 (Conv2D)	(None, 14, 14, 512)	2359808
block5_pool (MaxPooling2D)	(None, 7, 7, 512)	0
=====		
Total params: 14714688 (56.13 MB)		
Trainable params: 14714688 (56.13 MB)		
Non-trainable params: 0 (0.00 Byte)		

```
In [ ]: # Make the specified layers non-trainable
        for layer in vgg.layers[:-3]:
            layer.trainable = False

        vgg.summary()
```

Model: "vgg16"

Layer (type)	Output Shape	Param #
=====		
input_1 (InputLayer)	[(None, 224, 224, 3)]	0
block1_conv1 (Conv2D)	(None, 224, 224, 64)	1792
block1_conv2 (Conv2D)	(None, 224, 224, 64)	36928
block1_pool (MaxPooling2D)	(None, 112, 112, 64)	0
block2_conv1 (Conv2D)	(None, 112, 112, 128)	73856
block2_conv2 (Conv2D)	(None, 112, 112, 128)	147584
block2_pool (MaxPooling2D)	(None, 56, 56, 128)	0
block3_conv1 (Conv2D)	(None, 56, 56, 256)	295168
block3_conv2 (Conv2D)	(None, 56, 56, 256)	590080
block3_conv3 (Conv2D)	(None, 56, 56, 256)	590080
block3_pool (MaxPooling2D)	(None, 28, 28, 256)	0
block4_conv1 (Conv2D)	(None, 28, 28, 512)	1180160
block4_conv2 (Conv2D)	(None, 28, 28, 512)	2359808
block4_conv3 (Conv2D)	(None, 28, 28, 512)	2359808
block4_pool (MaxPooling2D)	(None, 14, 14, 512)	0
block5_conv1 (Conv2D)	(None, 14, 14, 512)	2359808
block5_conv2 (Conv2D)	(None, 14, 14, 512)	2359808
block5_conv3 (Conv2D)	(None, 14, 14, 512)	2359808
block5_pool (MaxPooling2D)	(None, 7, 7, 512)	0
=====		
Total params: 14714688 (56.13 MB)		
Trainable params: 4719616 (18.00 MB)		
Non-trainable params: 9995072 (38.13 MB)		

```
In [ ]: # Flattening the layer and adding custom Dense layers
x = Flatten()(vgg.output)

# Adding a fully connected layer with ReLU activation and He normal initializer
x = Dense(1024, activation='relu', kernel_initializer='he_normal')(x)
x = Dropout(0.5)(x) # Adding dropout for regularization

x = Dense(512, activation='relu', kernel_initializer='he_normal')(x)
```

```

x = Dropout(0.5)(x) # Adding dropout for regularization

# Adding the output layer with softmax activation
# Note: Adjust the number of units to match the number of classes you have
output = Dense(classes, activation='softmax', kernel_initializer='he_normal')(x)

# Creating the model
model = Model(inputs=vgg.input, outputs=output)

# Compile the model
model.compile(loss='categorical_crossentropy',
              optimizer=tf.keras.optimizers.Adam(learning_rate=0.0001, beta_1=0.9,
              metrics=['accuracy']))

# Model summary to see all layers
model.summary()

```

```

In [ ]: # File path for the model checkpoint
cnn_path = '/content/FER_2013_Emotion_Detection/VGG16_Transfer_Learning'
name = 'VGG16_Transfer_Learning.keras'
chk_path = os.path.join(cnn_path, name)

# Callback to save the model checkpoint
checkpoint = ModelCheckpoint(filepath=chk_path,
                             save_best_only=True,
                             verbose=1,
                             mode='min',
                             monitor='val_loss')

# Callback for early stopping
earlystop = EarlyStopping(monitor='val_loss',
                           min_delta=0,
                           patience=3,
                           verbose=1,
                           restore_best_weights=True)

# Callback to reduce learning rate
reduce_lr = ReduceLROnPlateau(monitor='val_loss',
                               factor=0.2,
                               patience=6,
                               verbose=1,
                               min_delta=0.0001)

# Callback to log training data to a CSV file
csv_logger = CSVLogger(os.path.join(cnn_path, 'training.log'))

# Aggregating all callbacks into a list
callbacks = [checkpoint, earlystop, reduce_lr, csv_logger] # Adjusted as per your

train_steps_per_epoch = train_generator.samples // train_generator.batch_size + 1
# validation_steps_epoch = validation_generator.samples // validation_generator.bat
test_steps_epoch = test_generator.samples // test_generator.batch_size + 1

```

```

In [ ]: history = model.fit(train_generator,
                             steps_per_epoch=train_steps_per_epoch,

```

```
validation_data=test_generator,
validation_steps=test_steps_epoch,
epochs=5,
callbacks = callbacks)
```

Epoch 1/5

```
449/449 [=====] - ETA: 0s - loss: 1.5836 - accuracy: 0.3833
Epoch 1: val_loss improved from inf to 1.33543, saving model to /content/FER_2013_Emotion_Detection/VGG16_Transfer_Learning/VGG16_Transfer_Learning.keras
449/449 [=====] - 519s 1s/step - loss: 1.5836 - accuracy: 0.3833 - val_loss: 1.3354 - val_accuracy: 0.4944 - lr: 1.0000e-04
```

Epoch 2/5

```
449/449 [=====] - ETA: 0s - loss: 1.3667 - accuracy: 0.4799
Epoch 2: val_loss improved from 1.33543 to 1.24391, saving model to /content/FER_2013_Emotion_Detection/VGG16_Transfer_Learning/VGG16_Transfer_Learning.keras
449/449 [=====] - 466s 1s/step - loss: 1.3667 - accuracy: 0.4799 - val_loss: 1.2439 - val_accuracy: 0.5277 - lr: 1.0000e-04
```

Epoch 3/5

```
449/449 [=====] - ETA: 0s - loss: 1.2823 - accuracy: 0.5124
Epoch 3: val_loss improved from 1.24391 to 1.20064, saving model to /content/FER_2013_Emotion_Detection/VGG16_Transfer_Learning/VGG16_Transfer_Learning.keras
449/449 [=====] - 413s 920ms/step - loss: 1.2823 - accuracy: 0.5124 - val_loss: 1.2006 - val_accuracy: 0.5325 - lr: 1.0000e-04
```

Epoch 4/5

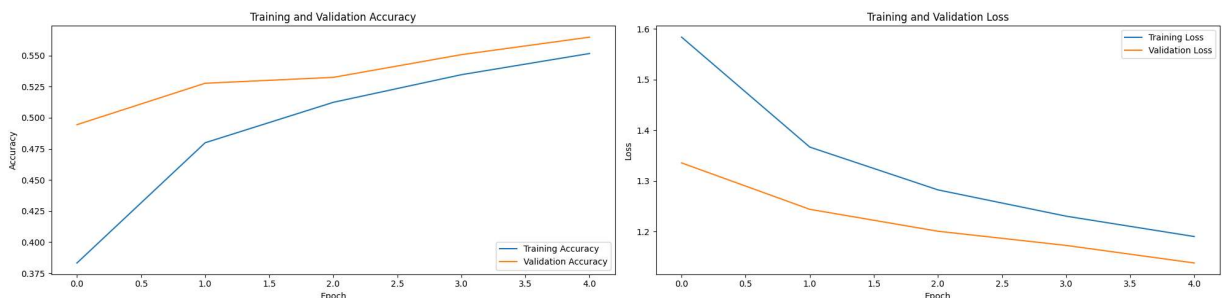
```
449/449 [=====] - ETA: 0s - loss: 1.2304 - accuracy: 0.5346
Epoch 4: val_loss improved from 1.20064 to 1.17266, saving model to /content/FER_2013_Emotion_Detection/VGG16_Transfer_Learning/VGG16_Transfer_Learning.keras
449/449 [=====] - 434s 968ms/step - loss: 1.2304 - accuracy: 0.5346 - val_loss: 1.1727 - val_accuracy: 0.5507 - lr: 1.0000e-04
```

Epoch 5/5

```
449/449 [=====] - ETA: 0s - loss: 1.1902 - accuracy: 0.5516
Epoch 5: val_loss improved from 1.17266 to 1.13800, saving model to /content/FER_2013_Emotion_Detection/VGG16_Transfer_Learning/VGG16_Transfer_Learning.keras
449/449 [=====] - 418s 931ms/step - loss: 1.1902 - accuracy: 0.5516 - val_loss: 1.1380 - val_accuracy: 0.5648 - lr: 1.0000e-04
```

Out[]: <keras.src.callbacks.History at 0x7a9e085b9720>

In []: plot_training_history(history_dict)



```
In [ ]: train_loss, train_accu = model.evaluate(train_generator)
test_loss, test_accu = model.evaluate(test_generator)
print("final train accuracy = {:.2f} , validation accuracy = {:.2f}".format(train_a
```

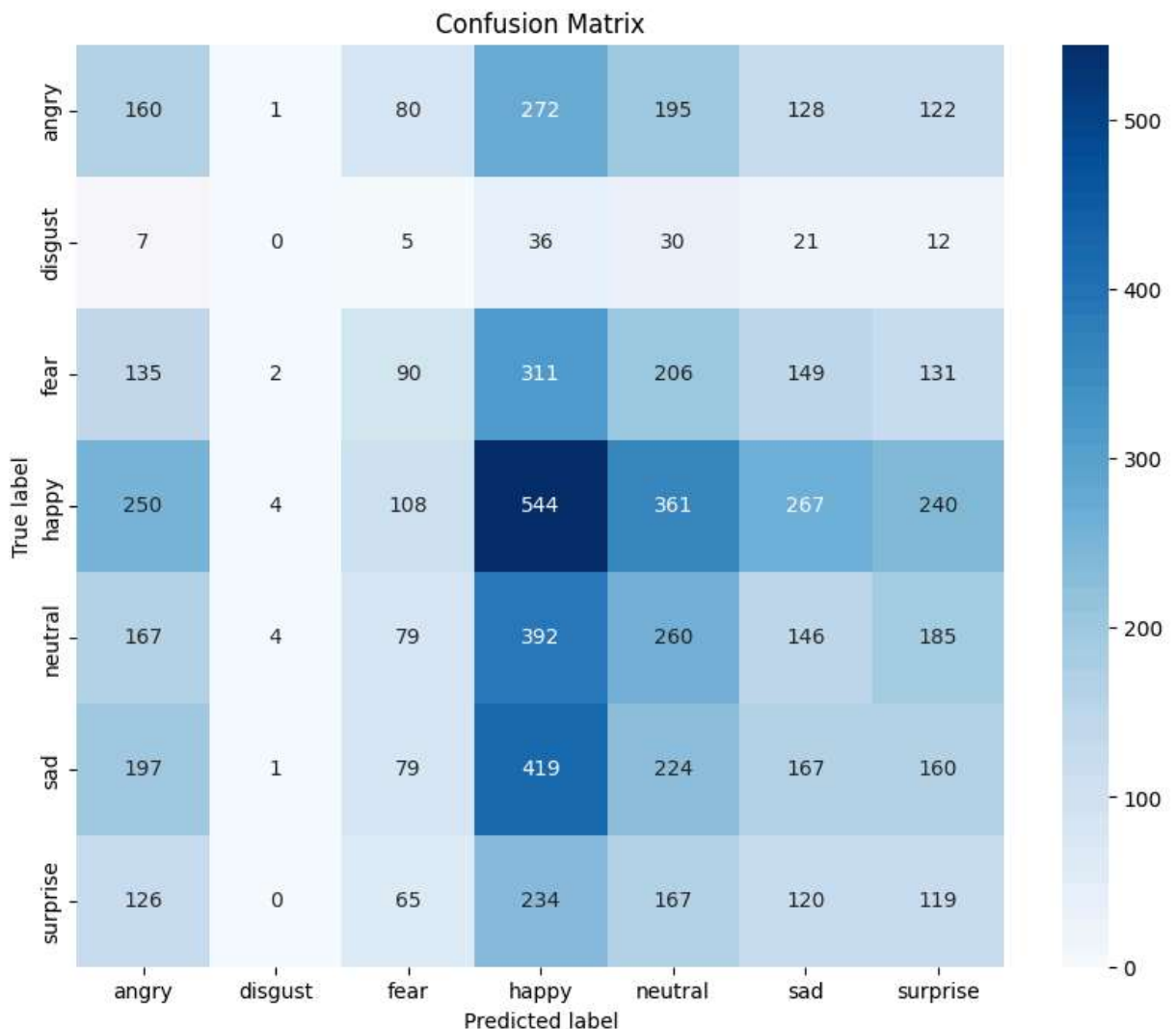
449/449 [=====] - 412s 916ms/step - loss: 1.1041 - accuracy: 0.5831
 113/113 [=====] - 28s 248ms/step - loss: 1.1380 - accuracy: 0.5648
 final train accuracy = 58.31 , validation accuracy = 56.48

```
In [ ]: # Assuming your true_classes and predicted_classes are already defined
true_classes = test_generator.classes
predicted_classes = np.argmax(model.predict(test_generator, steps=np.ceil(test_generator.steps_in_epoch)), axis=-1)
class_labels = list(test_generator.class_indices.keys())

# Generate the confusion matrix
cm = confusion_matrix(true_classes, predicted_classes)

# Plotting with seaborn
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=class_labels, yticklabels=class_labels)
plt.title('Confusion Matrix')
plt.ylabel('True label')
plt.xlabel('Predicted label')
plt.show()
```

113/113 [=====] - 27s 242ms/step



```
In [ ]: # Printing the classification report
report = classification_report(true_classes,
                              predicted_classes,
                              target_names=class_labels,
                              zero_division=0)
print("Classification Report:\n", report)
```

```
Classification Report:
              precision    recall  f1-score   support

   angry         0.15        0.17        0.16         958
  disgust         0.00        0.00        0.00         111
    fear         0.18        0.09        0.12        1024
   happy         0.25        0.31        0.27        1774
  neutral         0.18        0.21        0.19        1233
    sad          0.17        0.13        0.15        1247
  surprise         0.12        0.14        0.13         831

 accuracy                   0.19         7178
 macro avg         0.15        0.15        0.15         7178
 weighted avg         0.18        0.19        0.18         7178
```

Model 4 : Transfer Learning - ResNet50

```
In [ ]: train_dir = '/content/train'
test_dir = '/content/test'

# Data augmentation configuration for training data
train_datagen = ImageDataGenerator(rescale=1/255,           # Rescale pixel va
                                  rotation_range=40,        # Randomly rotate i
                                  width_shift_range=0.2,     # Randomly horizont
                                  height_shift_range=0.2,    # Randomly vertical
                                  shear_range=0.2,          # Apply shearing tr
                                  zoom_range=0.1,           # Randomly zoom ima
                                  horizontal_flip=True,      # Randomly flip ima
                                  fill_mode='nearest'        # Strategy used for
                                  )

# Rescaling for validation/test data (without further data augmentation)
test_datagen = ImageDataGenerator(
    rescale=1/255                                           # Rescale pixel val

)

# Creating data generators for training
train_generator = train_datagen.flow_from_directory(
    train_dir,
    target_size=(224, 224),
    color_mode='rgb',
    class_mode='categorical',
    batch_size=32
)

# Creating data generators for testing/validation
```

```
test_generator = test_datagen.flow_from_directory(
    test_dir,
    target_size=(224, 224),      # R
    color_mode='rgb',           # I
    class_mode='categorical',    # F
    batch_size=32               # S
)
```

Found 28709 images belonging to 7 classes.

Found 7178 images belonging to 7 classes.

Introducing Class wieghts for imbalanced data

```
In [ ]: # Extract class labels for all instances in the training dataset
classes = np.array(train_generator.classes)

# Calculate class weights to handle imbalances in the training data
# 'balanced' mode automatically adjusts weights inversely proportional to class fre
class_weights = compute_class_weight(
    class_weight='balanced', # Strategy to balance classes
    classes=np.unique(classes), # Unique class labels
    y=classes # Class labels for each instance in the training dataset
)

# Create a dictionary mapping class indices to their calculated weights
class_weights_dict = dict(enumerate(class_weights))

# Output the class weights dictionary
print("Class Weights Dictionary:", class_weights_dict)
```

Class Weights Dictionary: {0: 1.0266046844269623, 1: 9.406618610747051, 2: 1.0010460615781582, 3: 0.5684387684387684, 4: 0.8260394187886635, 5: 0.8491274770777877, 6: 1.293372978330405}

```
In [ ]: input_shape = (224, 224, 3)

base_model = ResNet50V2(include_top=False, weights='imagenet', input_shape=input_sh

model = Sequential([
    base_model,
    BatchNormalization(),
    GlobalAveragePooling2D(),
    Dense(512, activation='relu'),
    Dropout(0.1),
    Dense(256, activation='relu'),
    Dropout(0.1),
    Dense(128, activation='relu'),
    Dropout(0.1),
    Dense(7, activation='softmax')
])

optimizer = Adamax(learning_rate=0.0001)
```



```
model.compile(optimizer=optimizer, loss='categorical_crossentropy', metrics=['accuracy'])
model.summary()
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
resnet50v2 (Functional)	(None, 7, 7, 2048)	23564800
batch_normalization_2 (Batch Normalization)	(None, 7, 7, 2048)	8192
global_average_pooling2d_2 (GlobalAveragePooling2D)	(None, 2048)	0
dense_11 (Dense)	(None, 512)	1049088
dropout_8 (Dropout)	(None, 512)	0
dense_12 (Dense)	(None, 256)	131328
dropout_9 (Dropout)	(None, 256)	0
dense_13 (Dense)	(None, 128)	32896
dropout_10 (Dropout)	(None, 128)	0
dense_14 (Dense)	(None, 7)	903
Total params: 24787207 (94.56 MB)		
Trainable params: 24737671 (94.37 MB)		
Non-trainable params: 49536 (193.50 KB)		

```
In [ ]: # File path for the model checkpoint
cnn_path = '/content/FER_2013_Emotion_Detection/ResNet50_Transfer_Learning'
name = 'ResNet50_Transfer_Learning.keras'
chk_path = os.path.join(cnn_path, name)

# Callback to save the model checkpoint
checkpoint = ModelCheckpoint(filepath=chk_path,
                             save_best_only=True,
                             verbose=1,
                             mode='min',
                             monitor='val_loss')

# Callback for early stopping
earlystop = EarlyStopping(monitor='val_loss',
                           min_delta=0,
                           patience=6,
                           verbose=1,
                           restore_best_weights=True)

# Callback to reduce learning rate
reduce_lr = ReduceLROnPlateau(monitor='val_loss',
```



```

        factor=0.2,
        patience=6,
        verbose=1,
        min_delta=0.0001)

# Callback to Log training data to a CSV file
csv_logger = CSVLogger(os.path.join(cnn_path, 'training.log'))

# Aggregating all callbacks into a list
callbacks = [checkpoint, earlystop, csv_logger] # Adjusted as per your use-case

train_steps_per_epoch = train_generator.samples // train_generator.batch_size + 1
# validation_steps_epoch = validation_generator.samples // validation_generator.batch_size + 1
test_steps_epoch = test_generator.samples // test_generator.batch_size + 1

```

```

In [ ]: train_history = model.fit(
        train_generator,
        steps_per_epoch=200,
        epochs=50,
        validation_data=test_generator,
        validation_steps=100,
        class_weight=class_weights_dict,
        callbacks = callbacks
    )

```

```

Epoch 1/50
200/200 [=====] - ETA: 0s - loss: 1.9149 - accuracy: 0.2067
Epoch 1: val_loss improved from inf to 1.83747, saving model to /content/FER_2013_Emotion_Detection/ResNet50_Transfer_Learning/ResNet50_Transfer_Learning.keras
200/200 [=====] - 139s 575ms/step - loss: 1.9149 - accuracy: 0.2067 - val_loss: 1.8375 - val_accuracy: 0.2750
Epoch 2/50
200/200 [=====] - ETA: 0s - loss: 1.8385 - accuracy: 0.2570
Epoch 2: val_loss improved from 1.83747 to 1.74910, saving model to /content/FER_2013_Emotion_Detection/ResNet50_Transfer_Learning/ResNet50_Transfer_Learning.keras
200/200 [=====] - 114s 570ms/step - loss: 1.8385 - accuracy: 0.2570 - val_loss: 1.7491 - val_accuracy: 0.3609
Epoch 3/50
200/200 [=====] - ETA: 0s - loss: 1.7841 - accuracy: 0.3027
Epoch 3: val_loss improved from 1.74910 to 1.64001, saving model to /content/FER_2013_Emotion_Detection/ResNet50_Transfer_Learning/ResNet50_Transfer_Learning.keras
200/200 [=====] - 113s 565ms/step - loss: 1.7841 - accuracy: 0.3027 - val_loss: 1.6400 - val_accuracy: 0.4172
Epoch 4/50
111/200 [=====>.....] - ETA: 43s - loss: 1.6820 - accuracy: 0.3395
5

```

```

In [ ]: plot_training_history(train_history)

```

```

In [ ]: train_loss, train_accu = model.evaluate(train_generator)
        test_loss, test_accu = model.evaluate(test_generator)
        print("final train accuracy = {:.2f} , validation accuracy = {:.2f}".format(train_accu, test_accu))

```

```

In [ ]: # Assuming your true_classes and predicted_classes are already defined
        true_classes = test_generator.classes
        predicted_classes = np.argmax(model.predict(test_generator, steps=np.ceil(test_generator.samples)), axis=-1)

```

```

class_labels = list(test_generator.class_indices.keys())

# Generate the confusion matrix
cm = confusion_matrix(true_classes, predicted_classes)

# Plotting with seaborn
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=class_labels, ytickl
plt.title('Confusion Matrix')
plt.ylabel('True label')
plt.xlabel('Predicted label')
plt.show()

```

```

In [ ]: # Printing the classification report
report = classification_report(true_classes,
                               predicted_classes,
                               target_names=class_labels,
                               zero_division=0)
print("Classification Report:\n", report)

```

AUC ROC plot for each class

```

In [ ]: true_labels = test_generator.classes
preds = model.predict(test_generator, steps=len(test_generator))
pred_labels = np.argmax(preds, axis=1)
classes=list(test_generator.class_indices.keys())

```

```

In [ ]: y_encoded = pd.get_dummies(true_labels).astype(int).values
preds_encoded = pd.get_dummies(pred_labels).astype(int).values

fpr = dict()
tpr = dict()
roc_auc = dict()
for i in range(7):
    fpr[i], tpr[i], _ = roc_curve(y_encoded[:,i], preds_encoded[:,i])
    roc_auc[i] = auc(fpr[i], tpr[i])

plt.figure(figsize=(10,5))
colors = ['#1f77b4', '#ff7f0e', '#2ca02c', '#d62728', '#9467bd', '#8c564b', '#e377c2']
for i, color in enumerate(colors):
    plt.plot(fpr[i], tpr[i], color=color, lw=2, label=f"ROC curve for {classes[i]}")

plt.plot([0, 1], [0, 1], 'k--', lw=2)
plt.xlim([0, 1])
plt.ylim([0, 1])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.legend(loc='lower right')

```

```

In [ ]:

```